

Project Report – OPPORTUNA

STUDENT OPPORTUNITY & APPLICATION TRACKER

Submitted by : Saumya Dixit

Registration Number: 25BAI11460

Branch: B.Tech Computer Science and Engineering

Semester: 3rd

COURSE : JAVA PROGRAMMING

1. INTRODUCTION

Students come across many internships, hackathons, scholarships and competitions during college. The information about these opportunities is usually available through different sources such as college groups, emails, websites and social media.

Because the information is scattered, it can become difficult to keep track of deadlines, remember which opportunities have already been applied for, and check whether a particular opportunity matches the student's skills.

OPPORTUNA is a Java-based terminal application developed to solve this problem. It allows students to maintain their opportunities and applications in one place.

The project does more than simply store information. It compares the student's skills with the skills required for an opportunity, identifies missing skills, calculates a match percentage and gives a priority score based on factors such as skill match, deadline and stipend.

The project also includes application tracking, deadline reminders, file handling, JDBC and JPA implementations.

2. PROBLEM STATEMENT

Students often find internship and other career opportunities from multiple sources. Managing these opportunities manually can result in missed deadlines, duplicate applications and difficulty in deciding which opportunities are most relevant.

The main problems are:

1. Opportunity information is scattered across different sources.
2. Students may forget important deadlines.
3. It is difficult to quickly compare personal skills with required skills.
4. Students may lose track of their application status.
5. A simple list of opportunities does not help in deciding which opportunity needs attention first.

Therefore, a simple system is required that can store opportunities, compare them with student skills and help manage applications.

3. OBJECTIVES

The main objectives of OPPORTUNA are:

1. To maintain a student profile and their skills.
2. To store internship, hackathon, scholarship and competition details.
3. To compare student skills with the required skills of an opportunity.
4. To calculate the percentage of matching skills.
5. To identify the skills that the student is currently missing.
6. To calculate a priority score for opportunities.
7. To keep track of applications and their current status.
8. To prevent duplicate active applications.
9. To provide reminders for approaching deadlines.
10. To demonstrate different Java programming concepts in one practical project.
11. To make the application executable directly from the command line.

4. PROPOSED SOLUTION

OPPORTUNA provides a terminal-based environment where a student can manage opportunities and applications.

The user first creates a profile by entering basic information such as name, course and skills. Opportunities can then be viewed or added to the system.

For every opportunity, the system compares the required skills with the student's skills. It calculates the percentage of matching skills and displays the skills that are missing.

The system also calculates a priority score by considering the skill match, deadline urgency and stipend. This helps the student organize opportunities according to their current situation.

After applying for an opportunity, the student can track its application status.

The application is divided into different modules so that the user interface, business logic and data storage are handled separately.

5. WHY I CHOSE THIS PROJECT

I wanted to develop a project that could be related to an actual problem faced by students instead of creating only a basic management system.

Students regularly search for internships, hackathons and other opportunities, but the information is usually spread across different platforms.

A normal CRUD application could store these opportunities, but I wanted the system to do something useful with the stored information. For this reason, I added skill matching and priority calculation.

For example, instead of only displaying the following information:

- ❖ Internship: Software Developer
- ❖ Required Skills: Java, SQL, Git

The system can also provide:

- ❖ Skill Match: 66%
- ❖ Missing Skill: Git
- ❖ Priority Score: Calculated by the system

This makes the project more useful and also provides an opportunity to apply different Java concepts together.

6. SYSTEM WORKFLOW

The basic workflow of OPPORTUNA is:

START



Create Student Profile



Load / Add Opportunities



View Opportunities



Compare Student Skills



Calculate Skill Match



Find Missing Skills



Calculate Opportunity Priority



Apply for Opportunity



Track Application Status



Check Deadline Reminders



Exit

[7. MAIN MODULES](#)

7.1 STUDENT PROFILE MODULE

This module stores basic information about the student.

The profile contains:

- ❖ Student name
- ❖ Course
- ❖ Skills

The skill matching module later uses the stored skills.

7.2 OPPORTUNITY MODULE

This module is responsible for maintaining opportunity information.

An opportunity can be an:

- Internship
- Hackathon
- Scholarship
- Competition

The opportunity contains details such as:

- Opportunity title
- Organization
- Required skills
- Deadline
- Stipend

7.3 SKILL MATCHING MODULE

This is one of the main features of the project.

The system compares the skills entered by the student with the skills required for an opportunity.

Before comparison, the skill names are normalized so that different cases are treated as the same skill.

For example:

Java

JAVA

java

are considered the same skill.

The match percentage is calculated using:

Match Percentage =

$(\text{Number of Matched Skills} / \text{Total Required Skills}) \times 100$

Example:

Student Skills:

Java

Python

SQL

Required Skills:

Java

SQL

Git

OOP

Matched Skills:

Java

SQL

Therefore:

Match Percentage = $(2 / 4) \times 100$

= 50%

The system also displays:

Missing Skills:

Git

OOP

7.4 OPPORTUNITY PRIORITY MODULE

The priority module uses multiple factors instead of simply displaying opportunities in the order in which they were added.

The main factors considered are:

1. Skill Match
2. Deadline Urgency
3. Stipend

A higher skill match contributes more to the priority score. Opportunities with approaching deadlines also receive greater urgency.

The final score is restricted to a maximum of 100.

The purpose of this score is to help the student organize opportunities. It is not meant to represent the absolute quality of an opportunity.

7.5 APPLICATION TRACKING MODULE

The application module allows the student to track applications.

Example application statuses include:

APPLIED

SHORTLISTED

INTERVIEW

SELECTED

REJECTED

The system also maintains the status history of an application.

7.6 DUPLICATE APPLICATION CHECK

The system checks whether an active application for the same opportunity already exists before creating a new application.

This helps prevent accidental duplicate applications.

For example, if a student has already applied to an opportunity, the system will not unnecessarily create another active application for the same opportunity.

7.7 DEADLINE REMINDER MODULE

The project includes a deadline reminder feature.

A separate worker checks opportunities that are approaching their deadlines.

Java's ExecutorService is used to manage the background task.

This feature demonstrates the practical use of multithreading in the project.

7.8 FILE STORAGE MODULE

The project supports local file-based storage.

This allows the main application to work without requiring a database for the basic demonstration.

The application creates a data directory containing files such as:

opportunities.dat

applications.dat

activity.log

The activity log can be used to record important operations performed by the application.

7.9 JDBC MODULE

The project includes a JDBC implementation for MySQL.

It demonstrates:

- Database connection
- INSERT operation
- SELECT operation
- UPDATE operation
- DELETE operation
- Prepared Statement

This module allows the project to work with a relational database.

7.10 JPA MODULE

The project also contains a JPA/Hibernate implementation.

JPA provides an object-oriented approach to database operations.

The implementation uses:

- Entity Manager
- Entity classes
- persistence.xml
- Hibernate

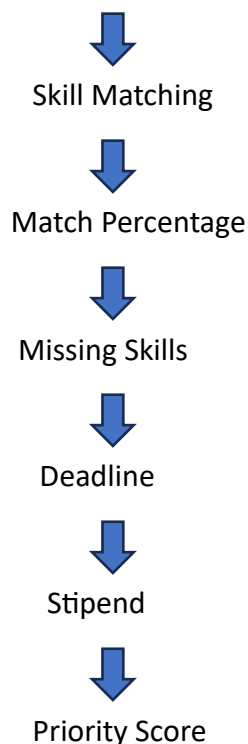
8. HIGHER-ORDER THINKING / DECISION LOGIC

The main reasoning part of OPPORTUNA is that the system does not only store information.

It processes the stored information and produces useful results.

The following process is used:

Student Skills + Opportunity Requirements



For example, if a student has Java and SQL but an opportunity requires Java, SQL, Git and OOP, the system identifies the two matching skills and the two missing skills.

The project therefore uses existing data to generate additional information that can help the student make decisions.

Another example is duplicate application prevention. Before creating an application, the program checks existing applications and verifies whether an active application already exists for the same opportunity.

9. DATA STRUCTURES USED

Different Java collection classes are used for different purposes.

9.1 ARRAYLIST

ArrayList is used for maintaining dynamic lists of opportunities and applications.

9.2 VECTOR

Vector is used for skill-related processing.

9.3 STACK

Stack is used for maintaining application status history.

For example:



The previous application states can be maintained using a stack.

9.4 HASHMAP

HashMap is used for quick access to calculated match information.

9.5 TREESET

TreeSet is used for maintaining unique and normalized skills.

9.6 PRIORITYQUEUE

Priority Queue is used to process opportunities according to their calculated priority.

10. OBJECT-ORIENTED PROGRAMMING

The project follows object-oriented programming principles.

10.1 ENCAPSULATION

The fields of important model classes are kept private and accessed through methods such as getters and setters.

This prevents direct and uncontrolled access to the data.

10.2 ABSTRACTION

Repository interfaces are used to hide the actual data storage implementation.

The service layer does not need to know whether the data is being stored in files or a database.

10.3 INTERFACES

Repository interfaces define common operations required for storing and retrieving data.

Different implementations can then provide the actual functionality.

Examples include:

- File Repository
- JDBC Repository
- JPA Repository

10.4 MODULAR DESIGN

The project is divided into packages based on responsibility.

For example:

Model , service , repository, exception, thread, annotation ,util

This makes the code easier to understand and maintain.

11. EXCEPTION HANDLING

Exception handling is used to handle invalid input and other runtime problems.

The project also contains custom exceptions such as:

Invalid Input Exception

Database Operation Exception

Instead of terminating the program immediately when an expected error occurs, the application displays an appropriate message and allows the user to continue.

12. FILE HANDLING

Java file handling is used for local data persistence. The project stores information in files inside the data directory.

The files include:

- opportunities.dat
- applications.dat
- activity.log

This allows the application to preserve information between different executions of the program.

13. MULTITHREADING

- Multithreading is used in the deadline reminder feature.
- The main application and reminder task can be handled separately.
- ExecutorService is used for managing the background task.
- Synchronization is also used where shared repository or file operations need protection.
- This demonstrates how concurrency can be applied to a practical feature instead of using threads only as a theoretical example.

14. ANNOTATIONS AND REFLECTION

- A custom annotation named @Feature is used in the project.
- Selected classes or methods can be marked using this annotation.
- Reflection is then used to inspect the classes at runtime and identify the annotated features.
- This demonstrates how Java can inspect class information during program execution.

15. DATABASE IMPLEMENTATION

The project provides two database approaches.

15.1 JDBC

JDBC provides direct communication between Java and MySQL.

The project demonstrates:

- INSERT
- SELECT

- UPDATE
- DELETE

Prepared Statement is used for parameterized queries.

15.2 JPA

JPA provides object-relational mapping between Java objects and database tables.

The project uses:

- Entity Manager
- Entity classes
- persistence.xml
- Hibernate

This gives the project both direct JDBC and ORM-based database implementations.

16. PROJECT STRUCTURE

The project is organized as follows:

OPPORTUNA



pom.xml



README.md



PROJECT_REPORT.md



schema.sql



src



main



java



com



opportuna



annotation



app



exception



model



repository



file



jdbc



jpa



service



thread



util



resources



META-INF



persistence.xml

17. TECHNOLOGIES USED

Programming Language:

Java

Build Tool:

Maven

Database:

MySQL

Database Connectivity:

DBC

ORM:

JPA / Hibernate

Other Java Features:

- OOP
- Collections
- Exception Handling
- File I/O
- Multithreading
- Synchronization
- Annotations
- Reflection

18. REQUIREMENTS

For running the basic terminal version:

- Java 17 or newer
- Maven 3.9 or newer

MySQL is not required for the basic file-based version.

MySQL is required only when testing the JDBC or JPA database implementation.

19. PROJECT EXECUTION

The project is designed to be executable through the command line.

After cloning the repository, the project can be compiled using:

```
mvn clean compile
```

The application can then be started using:

```
mvn exec:java
```

The main application does not require a GUI or an IDE to run.

The README file contains the setup and execution instructions so that the project can be run without previous knowledge of the project.

20. TESTING

The project was tested using the command line.

The following operations were checked:

1. Starting the application.
2. Creating a student profile.
3. Loading sample opportunities.
4. Viewing opportunities.
5. Adding opportunities.
6. Comparing student skills.
7. Calculating match percentage.
8. Identifying missing skills.
9. Calculating priority.
10. Creating an application.
11. Viewing application records.
12. Checking application status.
13. Running deadline reminders.
14. Checking file persistence.

The JDBC and JPA components require a properly configured MySQL environment before they can be tested.

21. SAMPLE USE CASE

Consider a student whose skills are:

Java

Python

SQL

Suppose an internship requires:

Java

SQL

Git

OOP

The application processes the information and produces:

Matched Skills:

Java

SQL

Missing Skills:

Git

OOP

Skill Match:

50%

The opportunity priority is then calculated using the skill match along with deadline and stipend information.

The student can then decide whether to apply and track the application through the system.

22. ADVANTAGES

The main advantages of OPPORTUNA are:

1. It solves a practical student problem.
2. It combines several Java concepts in one project.
3. It provides skill matching instead of only storing data.
4. It identifies missing skills.
5. It helps organize opportunities according to priority.

6. It tracks application status.
7. It prevents duplicate active applications.
8. It provides deadline reminders.
9. It supports local file storage.
10. It includes JDBC and JPA implementations.
11. It can be executed through the command line.

23. LIMITATIONS

The current version has some limitations.

1. Skill matching is based on normalized skill names and does not understand all possible relationships between similar skills.
2. The basic version uses local files instead of a remote database.
3. Deadline reminders run as part of the application rather than as a separate notification service.
4. The application currently uses a terminal interface.
5. External internship websites are not directly connected to the application.

24. FUTURE SCOPE

The project can be improved in the future by adding:

1. A web-based interface.
2. Login and authentication for multiple students.
3. Automatic import of opportunities.
4. API integration with opportunity platforms.
5. Email or mobile notifications.
6. More advanced skill matching.
7. Application analytics.
8. Resume-based skill extraction.
9. Personalized opportunity recommendations.
10. Cloud database support.

25. CONCLUSION

OPPORTUNA was developed with the idea of making opportunity management easier for students.

The project goes beyond basic data storage by processing student and opportunity information to calculate skill matches, identify missing skills and generate a priority score.

Application tracking, duplicate checking and deadline reminders provide additional practical functionality.

From the programming point of view, the project brings together several Java concepts including object-oriented programming, collections, exception handling, file handling, multithreading, synchronization, annotations, reflection, JDBC and JPA.

The project is also designed to run through the command line, which makes it suitable for terminal-based evaluation.

Overall, OPPORTUNA demonstrates how different Java concepts can be combined to solve a practical problem through a single application.

26. REFERENCES

1. Java Documentation
2. MySQL Documentation
3. JDBC Documentation
4. JPA Documentation
5. Hibernate Documentation
6. Maven Documentation

27. PROJECT OUTCOME

The final outcome of the project is a terminal-based Java application that allows a student to maintain opportunities, compare required skills with personal skills, track applications and monitor deadlines.

The project also demonstrates the practical implementation of multiple Java programming concepts rather than using them only as isolated examples.